

Pharma DevOps

Phase 1 -- Setup Guide

Windows 10/11 with WSL2 (Recommended) or PowerShell Native

GOAL

Deploy 5 microservices (auth-service, api-gateway, drug-catalog-service, notification-service, pharma-ui) on AWS EKS v1.32 via ArgoCD. No CI/CD yet -- that comes in Phase 2.

Services Deployed in Phase 1

Service	Stack	Port	Purpose
auth-service	Spring Boot	8081	Login, JWT, user management
api-gateway	Spring Boot	8080	Routing + JWT validation
drug-catalog-service	Spring Boot	8082	CRUD demo -- drugs, dosages
notification-service	Node.js	3000	Email notifications
pharma-ui	React/Nginx	80	Frontend UI

PREREQUISITES (AWS)

- AWS Account (brand new) with AdministratorAccess
- AWS CLI v2 configured with Access Key + Secret Key
- Project code cloned from GitHub: pharma-devops repo

PHASE ROADMAP

Phase 1 (This guide)	Infrastructure + Deploy on EKS via ArgoCD
Phase 2	GitHub Actions CI/CD -- automate build + deploy
Phase 3	Monitoring -- Prometheus + Grafana dashboards

PART A -- Install WSL2 (Ubuntu on Windows)

TIP WSL2 is strongly recommended. All bash commands in this guide work identically inside WSL2. If you use WSL2, you follow the same commands as Linux/Mac.

A1 Enable WSL2

Open PowerShell as Administrator (right-click Start → Windows PowerShell (Admin)):

```
# Install WSL2 with Ubuntu (default)
wsl --install

# Restart your PC when prompted
# After restart, Ubuntu opens automatically
# Set a username and password when asked
```

TIP If WSL2 was already installed, update it: `wsl --update`

A2 Verify WSL2 is Running Ubuntu

```
# In PowerShell
wsl --list --verbose
# Should show: Ubuntu Running 2

# Open Ubuntu terminal from Start menu
# Or type 'ubuntu' in PowerShell
```

A3 Install Docker Desktop and Enable WSL2 Backend

- Download Docker Desktop from docker.com/products/docker-desktop
- Run the installer -- accept defaults
- After install: open Docker Desktop
- Go to Settings → Resources → WSL Integration
- Toggle ON for Ubuntu
- Click Apply & Restart

```
# Verify Docker works inside WSL2 Ubuntu terminal
docker --version
# Should show: Docker version 25.x.x
```

PART B -- Install All Tools (Inside WSL2 Ubuntu Terminal)

TIP Open the Ubuntu app from Start Menu. Run ALL remaining commands in this Ubuntu terminal -- not PowerShell.

B1 Update Package Manager

```
sudo apt update && sudo apt upgrade -y
```

B2 Install AWS CLI v2

```
curl 'https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip' -o awscliv2.zip
sudo apt install unzip -y
unzip awscliv2.zip
sudo ./aws/install
rm -rf awscliv2.zip aws/

aws --version
# aws-cli/2.x.x
```

B3 Install Terraform via tfenv

```
git clone --depth=1 https://github.com/tfutils/tfenv.git ~/.tfenv
echo 'export PATH="$HOME/.tfenv/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc
tfenv install 1.7.5
tfenv use 1.7.5

terraform --version
# Terraform v1.7.5
```

B4 Install kubectl

```
curl -LO "https://dl.k8s.io/release/$(curl -sL https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
rm kubectl

kubectl version --client --short
```

B5 Install Helm

```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
```

```
helm version --short
```

B6 Install Java 17 and Node.js 18

```
# Java 17
sudo apt install openjdk-17-jdk -y
java --version

# Node.js 18 via nvm
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.bashrc
nvm install 18
nvm use 18
nvm alias default 18
node --version
# v18.x.x
```

B7 Install Git and jq

```
sudo apt install git jq -y
git --version && jq --version
```

B8 Fix Line Endings (Important for Windows)

```
# Prevent Windows CRLF line endings from breaking shell scripts
git config --global core.autocrlf false
git config --global core.eol lf
```

WARNING Always clone the project inside WSL2 home (~/) not on the Windows drive (/mnt/c/). The Windows filesystem is slow inside WSL2 and can cause build errors.

B9 Clone the Project Inside WSL2

```
# Inside Ubuntu terminal -- clone to WSL2 home directory
cd ~
git clone https://github.com/YOUR_ORG/pharma-devops.git
cd pharma-devops
```

PART C -- AWS Account Setup

2.1 Create IAM Admin User

In the AWS Console (browser -- same on all OS):

- Go to IAM → Users → Create user
- Username: devops-admin
- Attach policy: AdministratorAccess
- Create Access Key (type: CLI) → Download CSV

C1 Configure AWS CLI (inside WSL2 Ubuntu terminal)

```
aws configure
# AWS Access Key ID:      <paste from CSV>
# AWS Secret Access Key:  <paste from CSV>
# Default region name:    us-east-1
# Default output format:  json

# Verify
aws sts get-caller-identity
# Returns: Account, UserId, Arn
```

C2 Set Account ID Variable

```
export AWS_ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
export AWS_REGION=us-east-1
echo $AWS_ACCOUNT_ID
```

TIP Add these two export lines to ~/.bashrc so they persist across terminal sessions.

PART D -- Provision AWS Infrastructure with Terraform

1.1 Create S3 Bucket for Terraform State (one-time)

```
aws s3api create-bucket --bucket pharma-tf-state --region us-east-1
aws s3api put-bucket-versioning \
  --bucket pharma-tf-state \
  --versioning-configuration Status=Enabled

# Block public access
aws s3api put-public-access-block \
  --bucket pharma-tf-state \
  --public-access-block-configuration \
    BlockPublicAcls=true,IgnorePublicAcls=true,\
    BlockPublicPolicy=true,RestrictPublicBuckets=true

# Enable encryption
aws s3api put-bucket-encryption \
  --bucket pharma-tf-state \
  --server-side-encryption-configuration '{"Rules":[{"ApplyServerSideEncryptionByDefault":{"SSEAlgorithm":"AES256"}}
```

1.2 Create DynamoDB Lock Table (one-time)

```
aws dynamodb create-table \
  --table-name pharma-tf-lock \
  --attribute-definitions AttributeName=LockID,AttributeType=S \
  --key-schema AttributeName=LockID,KeyType=HASH \
  --billing-mode PAY_PER_REQUEST \
  --region us-east-1
```

1.3 Create terraform.tfvars

WARNING Never commit this file to Git. It contains passwords.

Navigate to: ~/pharma-devops/terraform/envs/dev/

```
cd ~/pharma-devops/terraform/envs/dev

cat > terraform.tfvars << 'EOF'
db_password = "PharmaSecure#2024Dev!"
jwt_secret  = "pharma-jwt-super-secret-dev-key-min-32-chars"
EOF
```

1.4 Run Terraform

```
terraform init
terraform plan -out=tfplan
terraform apply tfplan
```

Takes 15-25 minutes -- EKS cluster creation is slowest

What gets created:

Resource	Details
VPC	10.0.0.0/16, 3 subnet tiers -- network isolation
EKS Cluster	pharma-dev, Kubernetes v1.32, 1x t2.small node
RDS PostgreSQL	db.t3.micro, pharmadb -- shared database
ECR Repositories	5 private repos -- one per microservice
IAM Roles	ESO role, node role -- least-privilege access
Secrets Manager	dev/pharma/db and dev/pharma/jwt -- no secrets in code

TIP One terraform apply creates ~30 AWS resources in dependency order. This is IaC -- repeatable, version-controlled infrastructure.

PART E -- Configure Cluster

E1 Connect kubectl to EKS

Connect kubectl to EKS

```
aws eks update-kubeconfig \
  --region us-east-1 \
  --name pharma-dev \
  --alias pharma-dev

# Verify -- should show 1 node in Ready state
kubectl get nodes
kubectl get nodes -o wide
```

E2 Install Cluster Add-ons

Install Helm Repos + Namespaces

```
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo add external-secrets https://charts.external-secrets.io
helm repo add argo https://argoproj.github.io/argo-helm
helm repo update

kubectl apply -f k8s/namespaces.yaml
```

Install NGINX Ingress Controller

```
helm upgrade --install ingress-nginx ingress-nginx/ingress-nginx \
  --namespace ingress-nginx --create-namespace \
  --values k8s/ingress/nginx-values.yaml --wait
```

Install External Secrets Operator (ESO)

```
export AWS_ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)

helm upgrade --install external-secrets external-secrets/external-secrets \
  --namespace kube-system \
  --set installCRDs=true \
  --set serviceAccount.annotations."eks\.amazonaws\.com/role-arn"="\
  "arn:aws:iam::${AWS_ACCOUNT_ID}:role/pharma-dev-eso-role" \
  --wait

kubectl apply -f k8s/external-secrets/cluster-secret-store.yaml
kubectl apply -f k8s/external-secrets/dev-external-secrets.yaml
```

Install ArgoCD


```
kubectl apply -f argocd/install/argocd-namespace.yaml
```

```
helm upgrade --install argocd argo/argo-cd \
  --namespace argocd \
  --set server.service.type=ClusterIP \
  --wait
```

```
# Get ArgoCD admin password
```

```
kubectl get secret argocd-initial-admin-secret \
  -n argocd \
  -o jsonpath='{.data.password}' | base64 -d && echo
```

E3 Initialize Database Schemas

```
export RDS_ENDPOINT=$(terraform output -raw rds_endpoint)

kubectl run psql-init --rm -it \
  --image=postgres:15-alpine --namespace=dev \
  --env="PGPASSWORD=PharmaSecure#2024Dev!" \
  -- psql -h ${RDS_ENDPOINT} -U pharmaadmin -d pharmadb \
  -c "CREATE SCHEMA IF NOT EXISTS auth;
      CREATE SCHEMA IF NOT EXISTS drug_catalog;
      GRANT ALL ON SCHEMA auth, drug_catalog TO pharmaadmin;
      SELECT schema_name FROM information_schema.schemata;"
```

PART F -- Build Docker Images and Deploy

F1 Login to ECR and Build All Images

Login to ECR

```
export REGISTRY=${AWS_ACCOUNT_ID}.dkr.ecr.us-east-1.amazonaws.com
aws ecr get-login-password --region us-east-1 | \
  docker login --username AWS --password-stdin ${REGISTRY}
```

Build and Push All Services (build-all.sh)

```
#!/bin/bash
set -e
TAG='v1.0.0'
SERVICES=(auth-service api-gateway drug-catalog-service notification-service pharma-ui)

for svc in "${SERVICES[@]"; do
  echo '==== Building: '$svc' ====='
  docker build -t ${REGISTRY}/${svc}:${TAG} services/${svc}
  docker push ${REGISTRY}/${svc}:${TAG}
done
echo 'All 5 images pushed successfully!'

chmod +x build-all.sh
./build-all.sh
# Takes 15-20 min on first run (Maven downloads)
```

F2 Push Helm Charts to GitHub

```
# Create pharma-helm-charts repo on GitHub first, then:
cp -r ~/pharma-devops/helm-charts/* ~/pharma-helm-charts/

cd ~/pharma-helm-charts
git init && git checkout -b main
git add .
git commit -m "Initial Helm charts"
git remote add origin https://github.com/YOUR_ORG/pharma-helm-charts.git
git push -u origin main
```

Update image tags in values files

```
cd ~/pharma-helm-charts
REGISTRY=${AWS_ACCOUNT_ID}.dkr.ecr.us-east-1.amazonaws.com

for svc in auth-service api-gateway drug-catalog-service notification-service pharma-ui; do
```

```
FILE="envs/dev/values-`${svc}.yaml"
[ -f "$FILE" ] && sed -i "s|repository:.*|repository: ${REGISTRY}/${svc}|" "$FILE"
[ -f "$FILE" ] && sed -i "s|tag:.*|tag: v1.0.0|" "$FILE"
done
git add . && git commit -m 'Set image tags v1.0.0 for dev' && git push
```

F3 Configure ArgoCD and Deploy

Port-Forward ArgoCD UI

```
kubectl port-forward svc/argocd-server -n argocd 8080:443
# Open browser: https://localhost:8080
# Username: admin Password: (from step above)
```

Update ArgoCD App -- set your GitHub repo URL

Edit `argocd/apps/dev/pharma-dev-app.yaml` and change `repoURL`:

```
repoURL: https://github.com/YOUR_ORG/pharma-helm-charts.git
```

Apply ArgoCD Manifests

```
kubectl apply -f argocd/projects/pharma-project.yaml
kubectl apply -f argocd/apps/dev/pharma-dev-app.yaml
kubectl apply -f k8s/rbac/cluster-roles.yaml
kubectl apply -f k8s/rbac/dev-role.yaml
kubectl apply -f k8s/rbac/rolebindings.yaml
```

Sync in ArgoCD UI

- Open `https://localhost:8080`
- Click `pharma-dev` application
- Click `Sync` → `Synchronize`
- Watch pods appear in the `dev` namespace

```
# Watch from terminal
kubectl get pods -n dev -w
```

PART G -- Verify Everything Works

```
# All 5 pods should be Running
kubectl get pods -n dev

# Check services
kubectl get svc -n dev

# Test auth service
kubectl port-forward svc/auth-service -n dev 8081:8081 &

curl http://localhost:8081/actuator/health
# Expected: {"status":"UP"}

curl -X POST http://localhost:8081/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"admin@pharma.com","password":"Admin@123"}'
# Expected: JWT token in response
```

Windows-Specific Troubleshooting

Problem	Solution
Docker not found in WSL2	Docker Desktop → Settings → WSL Integration → enable Ubuntu
Scripts fail with lr errors	Run: git config --global core.autocrlf false, then re-clone
kubectl not found in WSL2	Install kubectl inside WSL2 -- Windows install doesn't carry over
Slow build performance	Clone project inside WSL2 home (~/) not on /mnt/c/
AWS CLI not found in WSL2	Install AWS CLI inside WSL2 (Part B, Step B2)
Permission denied on .sh	Run: chmod +x script.sh inside WSL2 terminal

Phase 1 Complete -- What Students Can Now See

- 5 microservices running on Kubernetes (auth, api-gateway, drug-catalog, notification, pharma-ui)
- ArgoCD managing deployments declaratively -- change a YAML file, ArgoCD syncs it
- Secrets pulled from AWS Secrets Manager (not hardcoded anywhere)
- NGINX Ingress routing all external traffic through one load balancer

Next phases:

- Phase 2:** GitHub Actions CI/CD -- automate the manual docker build + push
- Phase 3:** Monitoring -- Prometheus + Grafana dashboards
- Concepts:** Use this running cluster to demonstrate every K8s concept live